

Agentic AI for Optimizing FAISS HNSW Vector Database Parameters

Ryan Zhao

December 10, 2025

Abstract

Hierarchical Navigable Small World (HNSW) graphs are widely used for high-performance approximate nearest neighbor search in vector databases, but their performance depends on three coupled parameters (M , $efConstruction$, $efSearch$) that are difficult to tune by hand. We present an *agentic AI system* that uses a language model to propose FAISS HNSW configurations, evaluate them, and refine its strategy using logged experiments. The agent supports two workload profiles: a Knowledge + Reasoning database optimized for high recall (target ≥ 0.90) and a Memory + Reaction database optimized for low latency while maintaining recall around 0.70. On million-vector synthetic datasets, the system reliably finds acceptable configurations for both profiles within a small number of iterations, reducing manual tuning effort significantly.

1 Introduction

Vector databases have become essential infrastructure for modern AI applications, enabling efficient similarity search across high-dimensional embeddings for tasks such as retrieval-augmented generation (RAG), recommendation systems, and semantic search. Among various indexing techniques, Hierarchical Navigable Small World (HNSW) graphs [MY18] have emerged as a leading solution, offering an excellent balance between search accuracy and query latency.

FAISS (Facebook AI Similarity Search) [JDJ17, Res24] provides a high-performance implementation of HNSW, but achieving optimal performance requires careful tuning of three interdependent parameters:

- **M** : Graph connectivity, controlling the number of outgoing edges per node. Higher values improve recall but increase memory usage and build time.
- **$efConstruction$** : Beam width during index construction. Higher values produce better graph quality and recall, but significantly increase build time.
- **$efSearch$** : Beam width during query execution. Higher values improve recall but increase query latency.

The parameter space is high-dimensional and non-linear, with complex trade-offs between recall, latency, and memory. Traditional approaches rely on fixed heuristics, grid search, or manual expert tuning, all of which are computationally expensive, inflexible, and often fail to find optimal configurations for specific use cases.

This research introduces an **agentic AI system** that autonomously optimizes FAISS HNSW parameters through iterative experimentation and reasoning. Unlike static optimization methods, our agent adapts its strategy based on performance feedback, learns from historical experiments, and can optimize for different objectives depending on the application requirements.

2 Background and Related Work

HNSW (Hierarchical Navigable Small World) graphs [MY18] construct a multi-layer graph that supports logarithmic-time approximate nearest neighbor search. Their performance is highly sensitive

to three parameters (M , efConstruction , efSearch), which jointly control recall, latency, and memory footprint. FAISS [JDJ17, Res24] provides optimized CPU and GPU implementations of HNSW but offers little guidance on how to tune these parameters for particular workloads.

Automated hyperparameter optimization has been widely studied for machine learning models, using Bayesian optimization, evolutionary search, and reinforcement learning. In contrast, relatively few works target low-level database index parameters, and most still rely on black-box search. Recent advances in large language models (LLMs) have enabled *agentic AI systems* that can reason about objectives and constraints, making them a natural fit for steering such search processes rather than replacing them.

3 Methodology

Our agentic optimization system, implemented using LangGraph, operates as a stateful workflow. At each iteration the system (i) analyzes the dataset, (ii) proposes parameters, (iii) builds and evaluates an index, and (iv) decides whether and how to continue. Dataset analysis records size and dimensionality; parameter generation uses either an LLM or a heuristic fallback; evaluation measures recall, latency, and memory; and decision logic updates the best configuration and stops once a maximum number of experiments or performance targets are reached.

The optimization is structured into two phases. In the *recall phase*, the agent explores configurations that drive recall above a minimum workload-specific threshold (e.g., ≥ 0.90 for Knowledge + Reasoning and around 0.70 for Memory + Reaction). Once that target is met, it switches to a *latency phase* where it tries to reduce latency (primarily by lowering efSearch) while keeping recall above the same threshold. If recall degrades, the agent automatically returns to the recall phase.

We capture workload differences via simple database “profiles”. The Knowledge + Reasoning profile assumes that missing relevant results is costly and therefore prefers higher M and efSearch and is allowed higher latency and memory usage. The Memory + Reaction profile assumes tight latency and memory budgets, prefers smaller M and efSearch , and treats recall thresholds as minimal acceptability constraints rather than primary objectives.

The agent uses LLM-guided parameter proposals that learn from both recent trials in the current run and historical experiments from past runs. To optimize for expensive large-scale experiments, we eliminated the initial exploration phase by default and implemented index reuse between evaluation steps, reducing build time by approximately 50% for million-scale datasets. All trials are logged to JSONL files organized by database type and dataset size for cross-run learning and analysis.

4 Experiments and Results

4.1 Experimental Setup

Experiments were conducted using:

- FAISS version 1.7.4
- Synthetic datasets: 1M-vector scale, 128 dimension
- Query sets: 10,000 queries per dataset
- Evaluation metrics: Recall@10, true recall (brute-force for datasets up to 1M vectors), average query latency (ms per query), memory (GB), index size (MB)

4.2 Results on Million-Scale Datasets

We report results on synthetic 128-dimensional datasets with 10^6 vectors for both Knowledge + Reasoning and Memory + Reaction workloads. All metrics are computed using exact brute-force recall computation (IndexFlatL2) for accurate evaluation. The agent successfully achieved target performance metrics for both workload profiles through iterative optimization.

Knowledge + Reasoning Profile The agent optimized for high recall (target ≥ 0.90) and achieved excellent results. Across multiple optimization runs, the system consistently found configurations that met or exceeded the recall target. Representative results include:

- Configuration ($M = 40$, efConstruction = 2309, efSearch = 870): True recall@10 of 0.9995 with latency 0.63 ms
- Configuration ($M = 32$, efConstruction = 400, efSearch = 200): True recall@10 of 0.955 with latency 0.31 ms
- Configuration ($M = 16$, efConstruction = 300, efSearch = 500): True recall@10 of 0.908 with latency 0.25 ms

These results demonstrate that the agent successfully navigated the parameter space to achieve recall values consistently above the 0.90 target. The best configuration achieved near-perfect recall (0.9995) with sub-millisecond latency by automatically selecting higher connectivity ($M = 40$) and balanced search depth (efSearch 870).

Memory + Reaction Profile The agent optimized for low latency while maintaining recall around 0.70. The system successfully balanced these competing objectives:

- Configuration ($M = 24$, efConstruction = 200, efSearch = 50): True recall@10 of 0.691 with latency 0.065 ms
- Configuration ($M = 16$, efConstruction = 300, efSearch = 100): True recall@10 of 0.690 with latency 0.085 ms
- Configuration ($M = 16$, efConstruction = 300, efSearch = 120): True recall@10 of 0.7195 with latency 0.096 ms

The agent consistently found configurations achieving recall around 0.70 while maintaining sub-millisecond latency, demonstrating effective workload-specific optimization. The best configuration achieved recall of 0.691 with only 0.065 ms latency by selecting moderate connectivity ($M = 24$) and minimal search depth (efSearch 50) to prioritize speed while maintaining acceptable accuracy.

4.3 Summary of Workload-Specific Tuning

Table 1 summarizes the key quantitative results for the two workloads on the million-scale dataset we evaluated.

Workload	Dataset Size	M	efSearch	Recall@10	Latency (ms)
Knowledge + Reasoning	10^6	40	870	0.9995	0.63
Knowledge + Reasoning	10^6	32	200	0.955	0.31
Knowledge + Reasoning	10^6	16	500	0.908	0.25
Memory + Reaction	10^6	24	50	0.691	0.065
Memory + Reaction	10^6	16	100	0.690	0.085
Memory + Reaction	10^6	16	120	0.7195	0.096

Table 1: Best workload-specific configurations from experiment logs. All recall values are computed using exact brute-force search (IndexFlatL2) for accurate evaluation. The agent achieved ≥ 0.90 recall for Knowledge + Reasoning (reaching 0.9995) and recall around 0.70 for Memory + Reaction with sub-millisecond latency (0.065 ms best).

These results demonstrate that the agentic system successfully achieved its optimization objectives: the Knowledge + Reasoning profile consistently exceeded the 0.90 recall target (reaching up to 0.9995), while the Memory + Reaction profile achieved recall around 0.70 while maintaining sub-millisecond latency. Across 50 optimization runs with 5 iterations per experiment, the agent consistently found acceptable configurations, demonstrating efficient convergence. The agent automatically selected distinct parameter configurations that align with each workload’s qualitative goals, demonstrating effective workload-aware optimization.

5 Discussion

This work shows that agentic AI systems can effectively optimize low-level vector database parameters by combining structured workflows with LLM-based reasoning. Using a simple phase-aware exploration strategy and lightweight workload profiles, a single agent can navigate trade-offs between recall, latency, and memory, discovering distinct configurations for reasoning-oriented versus reaction-oriented workloads.

Several limitations remain. Large-scale experiments are computationally expensive, as each evaluation requires constructing an HNSW index and issuing thousands of queries, limiting exhaustive exploration for datasets beyond million scale or for systems requiring frequent re-tuning. A critical challenge arises from the reliance on brute-force exact search (IndexFlatL2) to compute ground-truth neighbors for recall estimation. Although feasible in controlled research settings, this becomes impractical in production environments where datasets may reach tens or hundreds of millions of vectors and where continuous or on-demand tuning is desired. Without access to true recall, the system must depend on approximations that introduce uncertainty. Practical solutions include caching ground truth for stable portions of the dataset, computing recall on representative samples, using high-efSearch configurations as proxy ground truth, or employing learned recall estimators that map observable metrics to recall predictions. These strategies reduce computational cost while maintaining reliable feedback for optimization.

Future work could address these limitations along several promising directions. Reinforcement learning (RL) provides a natural extension, enabling an agent to learn policies that map system state (parameter values, performance metrics, history) to updates through repeated optimization episodes. Such policies could internalize exploration-exploitation trade-offs and uncover non-obvious parameter relationships. Additional directions include multi-agent architectures with specialization across optimization tasks, broader FAISS index support (e.g., IVF and product quantization), GPU-accelerated evaluation pipelines, and real-time workload monitoring for continuous adaptation to shifting query patterns.

6 Conclusion

We presented an agentic AI system that autonomously tunes FAISS HNSW vector database parameters through iterative experimentation guided by language models. By structuring optimization into recall and latency phases with simple workload profiles, the system discovers distinct configurations for Knowledge + Reasoning and Memory + Reaction databases that reflect their different priorities.

On million-vector datasets, the agent achieves recall@10 values of at least 0.90 for Knowledge + Reasoning and around 0.70 with sub-millisecond latency for Memory + Reaction, across 50 experiments with 5 iterations each. These results demonstrate that a lightweight agent can approach expert-level tuning without exhaustive search, making automated parameter optimization practical for real-world deployments.

The framework’s modular design—separating measurement, reasoning, and decision logic—enables cross-run learning and provides a foundation for extending to other FAISS index types or real-world workloads like RAG and recommendation systems. For production use, the computational cost of ground truth computation can be addressed through strategies such as pre-computed caching, sampling-based evaluation, learned recall estimators, or progressive evaluation that reserves exact computation for final validation. The choice depends on dataset scale, query stability, and accuracy requirements, but these approaches enable production-ready systems that balance optimization quality with computational efficiency.

References

- [JDJ17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus, 2017.
- [MY18] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2018.

[Res24] Facebook AI Research. Faiss library. <https://github.com/facebookresearch/faiss>, 2024.
Accessed: 2025-11-10.